



Encontro 6

26/09/2026 | Observabilidade, IA aplicada e projeto final

Fechar o ciclo: da entrega à evidência

Encontro 6 • observabilidade, diagnóstico e apresentação final

Retomar

E4: executar e integrar.
E5: entregar e recuperar.
E6: observar o efeito da mudança.

Compreender

Logs, métricas e traces.
Indicadores, objetivos e alertas.
Incidente com causa comprovada.

Demonstrar

Instrumentação mínima.
Relatório do incidente.
Apresentação do mesmo projeto da equipe.

Uma entrega só gera aprendizado quando seu comportamento pode ser observado e explicado.

Quatro horas com espaço para as apresentações

Planejamento para até sete equipes • ajustar ao número real de grupos

0–80 min | Base

0–10: contexto.
10–40: sinais e instrumentação.
40–60: SLI/SLO e alertas.
60–80: diagnóstico de incidente.

80–180 min | Aplicar

80–95: intervalo.
95–125: demo guiada.
125–145: IA e revisão.
145–180: preparação das equipes.

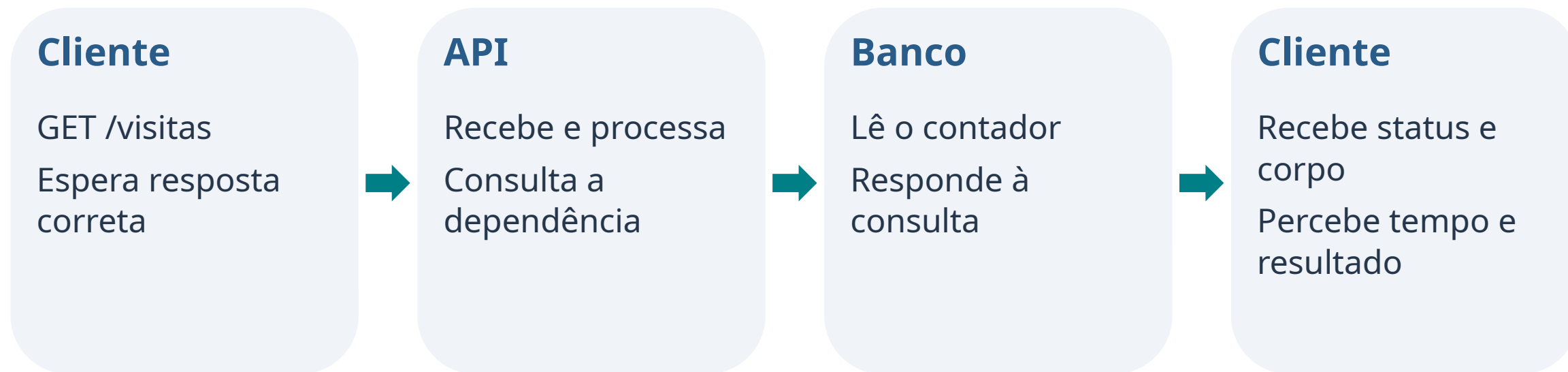
180–240 min | Fechar

180–236: até 7 blocos de 8 min.
Cada fala: 5–7 min.
236–240: síntese e submissão final.

O último encontro precisa de tempo para observar, preparar evidências e apresentar; não apenas exposição.

Comece pelo caminho que o usuário percorre

Exemplo conhecido: consultar o contador de visitas



Leia o fluxo: cada etapa tem um objeto, uma decisão e uma evidência.

Observe uma operação relevante; CPU baixa ou container Up não provam que ela funciona.

Monitoramento e observabilidade: capacidades próximas

Medir o conhecido e investigar o comportamento do sistema

Monitorar

Acompanhar condições escolhidas.

Ex.: erro e duração de uma rota.

Detectar desvio de um critério.

Investigar

Combinar sinais e contexto.

Explorar uma falha ainda não prevista.

Formular e testar hipóteses.

Instrumentar

Emitir dados úteis no código.

Preservar versão, tempo e contexto.

Coletar e consultar os sinais.

Ferramenta organiza dados; a instrumentação e o contrato da aplicação dão significado a eles.

Logs, métricas e traces: três perspectivas

Cada sinal responde melhor a um tipo de pergunta

Sinal	O que representa	Exemplo
Log	Registro de um evento com contexto.	Requisição terminou com 503 e código de erro.
Métrica	Medição agregada ao longo do tempo.	5 falhas em 20 requisições.
Trace	Caminho e duração de operações correlacionadas.	API aguardou uma chamada à dependência.

Combine perspectivas; não espere que um único sinal prove sozinho toda a causa.

Log estruturado: campos que apoiam o diagnóstico

Exemplo ilustrativo com valores fictícios • JSON facilita consulta

```
{ "timestamp": "2026-09-26T12:00:00Z",  
  "service": "api-visitas",  
  "version": "3.0.0-observabilidade",  
  "request_id": "exemplo-123",  
  "method": "GET", "route": "/visitas",  
  "status": 503, "duration_ms": 12.4,  
  "error_code": "ECONNREFUSED" }
```

Contexto

timestamp: quando.
version: qual código.
request_id: qual chamada.

Resultado

route e status: operação e resposta.
duration_ms: tempo no servidor.
error_code: pista da falha.

Um log útil permite localizar o evento sem expor dados que a investigação não precisa.

O que registrar — e o que não precisa ir para o log

Utilidade, retenção e acesso fazem parte do desenho

Registrar

Evento, hora e serviço.

Rota normalizada, status e duração.

Versão e identificador de correlação.

Evitar

Senha, token e cabeçalho de autorização.

Payload completo sem necessidade.

Dados pessoais e URLs sensíveis.

Controlar

Quem consulta e por quanto tempo.

Volume e nível dos eventos.
Separar produção de dados fictícios.

Mais texto não significa mais evidência; selecione contexto que ajude a responder a uma pergunta.

Identificação da requisição: ligar resposta e evento

No laboratório a API gera X-Request-Id para cada chamada

Resposta HTTP

Cabeçalho com o ID
Copiar o valor observado



Log da API

Mesmo request_id
Status, versão e duração



Investigação

Localizar aquele evento
Comparar com outros sinais

Leia o fluxo: cada etapa tem um objeto, uma decisão e uma evidência.

request_id é uma chave de correlação; sozinho não descreve o caminho distribuído.

Tipos de métrica: conte, meça e distribua

Escolher o tipo evita interpretações erradas

Tipo	Comportamento	Exemplo
Counter	Acumula eventos; pode zerar na reinicialização.	Total de respostas 503.
Gauge	Valor que pode subir e descer.	Memória usada ou fila atual.
Histograma	Conta observações em faixas de valores.	Duração das requisições.

Diga sempre o nome, a unidade, a população e o intervalo da medição.

De contador a taxa: exemplo resolvido

Mesma instância, sem reinício entre as duas leituras

Observação	Cálculo e interpretação
12:00:00 → total 100	Primeira leitura do counter.
12:00:10 → total 140	Foram concluídas 40 requisições no intervalo.
Taxa média	$(140 - 100) / 10 \text{ s} = 4 \text{ requisições/s.}$
Se o valor caiu para 3	Pode ter ocorrido reset; não calcular uma taxa negativa.

Counter é total; taxa é variação por tempo; reinícios mudam como a variação deve ser interpretada.

Média e percentil mostram coisas diferentes

Exemplo sintético: 100 requisições ordenadas por duração

95 chamadas × 20 ms



4 chamadas × 200 ms



1 chamadas × 2000 ms



RESULTADO

Média: 47 ms

p95: 20 ms

p99: 200 ms

Máximo: 2.000 ms

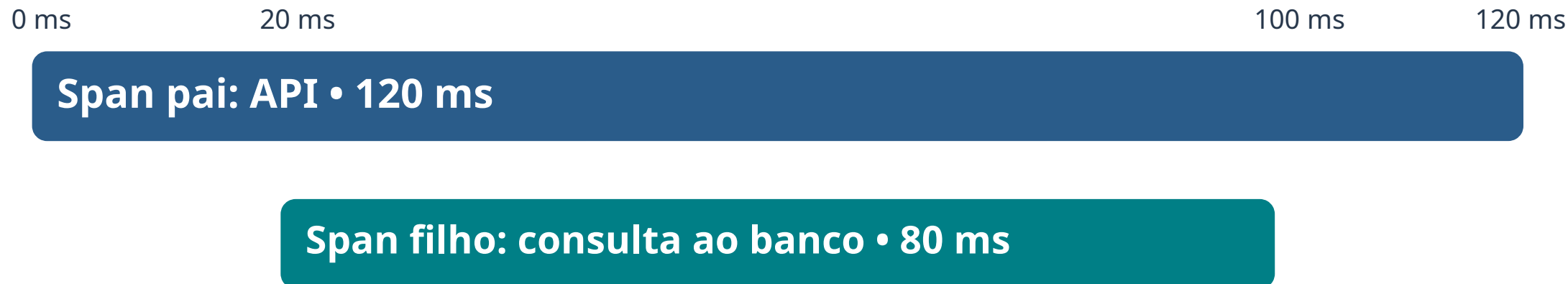
Comprimento das barras = quantidade de chamadas (total: 100).

Uma média baixa pode conviver com requisições lentas; percentil também depende da amostra e do método.

Trace: visualizar a duração e a relação entre operações

Diagrama conceitual • valores ilustrativos, não capturados no laboratório

Uma requisição • eixo horizontal: tempo desde o início

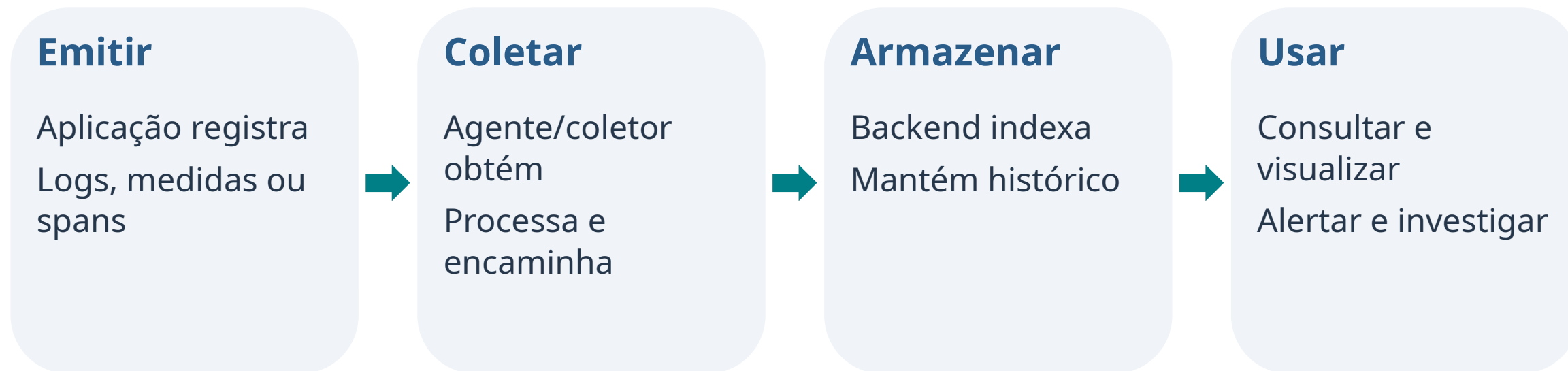


Os 80 ms estão dentro dos 120 ms: não somar pai + filho.

Spans mostram relações e intervalos; duração maior sugere onde investigar, não prova a causa sozinha.

Do código ao painel: como a telemetria chega à consulta

Separar instrumentação, transporte, armazenamento e uso



Leia o fluxo: cada etapa tem um objeto, uma decisão e uma evidência.

O dado precisa chegar à consulta com contexto; emitir um endpoint não cria automaticamente histórico.

Ferramentas: reconhecer a responsabilidade de cada uma

Exemplos de componentes • nenhuma plataforma paga é exigida

Componente	Papel típico	Na aula
Logs do Docker	Acessar a saída emitida pelo processo.	Usado na demonstração.
Prometheus	Coletar e consultar séries de métricas.	Formato de exposição e consulta conceitual.
Grafana	Visualizar dados de fontes configuradas.	Exemplo de finalidade de dashboard.
OpenTelemetry / Jaeger	Instrumentar/exportar; analisar traces, respectivamente.	Conceitos e diagrama.

Comece pela pergunta operacional; depois escolha como emitir, guardar e consultar o sinal.

Verificação | Interpretar os sinais

Depois dos conceitos e exemplos • explique a evidência

1. Situação

O counter está em 500. Isso significa 500 requisições por segundo?

2. Situação

Uma chamada tem request_id no log. Já temos um trace distribuído completo?

3. Situação

A média é 47 ms. Podemos afirmar que ninguém esperou dois segundos?

O próximo slide traz as três respostas explicadas.

Respostas comentadas | Interpretar os sinais

Gabarito • do conceito à decisão

1. Resposta

Não. É um total acumulado. Para taxa, calcule variação por tempo e trate resets.

2. Resposta

Não. Correlação local não substitui spans e propagação de contexto entre operações e serviços.

3. Resposta

Não. A média pode esconder cauda lenta; no exemplo há uma chamada de 2000 ms.

Diferencie o que foi observado, o que é hipótese e o que foi confirmado.

SLI: definir exatamente o que será medido

Indicador de nível de serviço ligado a uma experiência do usuário

População

GET /visitas elegíveis.
Janela declarada.
Origem da observação identificada.

Evento bom

Status 200 e total inteiro válido.
Falha de conexão também importa na visão do cliente.

Indicador

Chamadas boas / chamadas elegíveis.
Reportar volume e resultado.
Sem chamadas: sem amostra.

Um SLI é útil quando população, sucesso, janela e origem estão definidos.

SLI calculado: explicação passo a passo

Exemplo fictício de uma janela já encerrada

Dado	Cálculo / significado
Elegíveis	1000 chamadas de GET /visitas.
Boas	992 retornaram o contrato esperado.
Falhas	$1000 - 992 = 8$ chamadas não boas.
SLI de sucesso	$992 / 1000 \times 100 = 99,2\%$.

99,2% descreve esta população nesta janela; não é uma promessa para todas as situações.

SLI, SLO e SLA: medida, objetivo e compromisso

A meta deve fazer sentido para quem usa o serviço

Termo	Significado	Exemplo didático
SLI	Indicador observado.	99,2% de chamadas boas na janela.
SLO	Objetivo para indicador e janela definidos.	Pelo menos 99% no período acordado.
SLA	Compromisso formal com condições próprias.	Nível acordado e consequências previstas.

Medida observada, meta interna e compromisso formal são objetos diferentes.

Error budget: quanto da tolerância já foi consumido

Mesmo exemplo: 1000 chamadas • SLO 99% • 8 falhas

CONTRATO

1.000 operações

SLO: 99% boas

10 falhas permitidas

8 falhas observadas → 80% do orçamento usado

8 usadas

2 livres

$992 / 1.000 = 99,2\%$ boas

Objetivo cumprido nesta janela.

Orçamento permitido: 10 • consumido: 8 • restante: 2; a política define como isso orienta decisões.

Alertas: condição, impacto e ação

Exemplos ilustrativos; calibrar janela e volume para o serviço

Alerta	Problema ou mérito	Melhoria / ação
CPU > 80% em uma amostra	Pode não representar falha percebida.	Relacionar a saturação e impacto; observar duração.
Erros aumentaram na operação	Sinal próximo do resultado do usuário.	Definir janela, amostra mínima e destino.
Dependência indisponível	Contexto útil, mas pode duplicar alertas.	Agrupar por incidente e consultar runbook.
Sonda sem dados	Não é sucesso nem falha medida da aplicação.	Investigar a coleta e o acesso ao destino.

Alerta é um convite a agir com contexto, não apenas uma cor vermelha no painel.

Dashboard mínimo: quatro perguntas operacionais

Esboço didático • cada painel precisa de unidade, janela e contexto

TRÁFEGO

Chamadas por segundo
Volume na janela escolhida

ERROS

% de operações não boas
Contrato e denominador explícitos

LATÊNCIA

p95 em milissegundos
População e janela definidas

SATURAÇÃO

Uso / limite do recurso
Ex.: conexões usadas / disponíveis

Quatro sinais úteis: tráfego, erros, latência e saturação; associe-os à operação que importa.

Verificação | Indicadores e decisões

Depois dos conceitos e exemplos • explique a evidência

1. Situação

Em 1000 chamadas, 992 boas e SLO 99%, quanto orçamento foi consumido?

2. Situação

Não houve chamadas. Podemos exibir 100% de sucesso?

3. Situação

50 alertas por dia são ignorados. Basta desligar todos para resolver?

O próximo slide traz as três respostas explicadas.

Respostas comentadas | Indicadores e decisões

Gabarito • do conceito à decisão

1. Resposta

A tolerância é 10 falhas. Oito foram consumidas: 80% do orçamento, restando duas nesta população.

2. Resposta

Não. O denominador é zero: faltam observações. Indique ausência de amostra e confira a coleta.

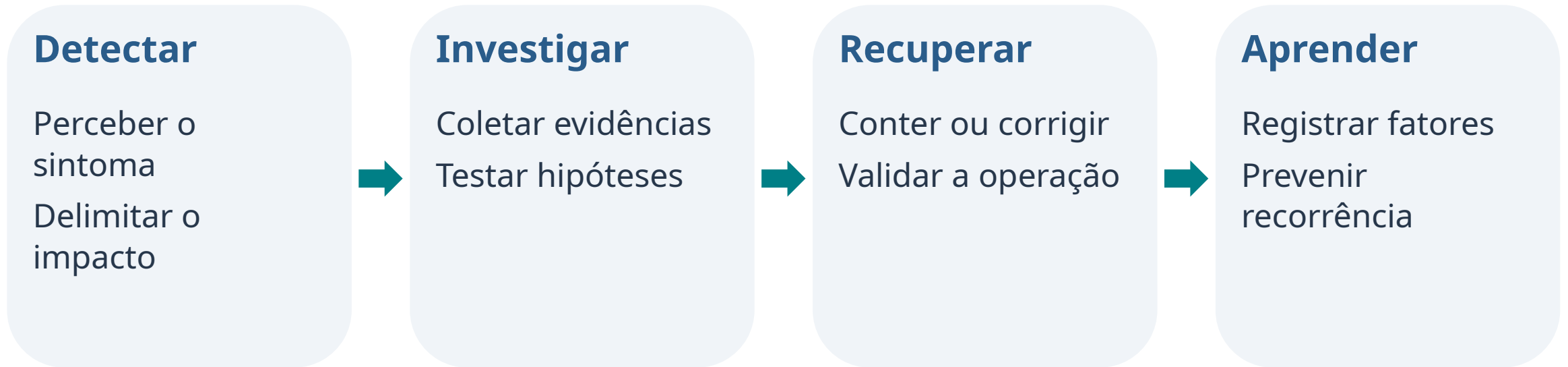
3. Resposta

Não. Priorize sinais ligados a impacto, agrupe duplicados e defina ação, dono e janela de cada regra.

Diferencie o que foi observado, o que é hipótese e o que foi confirmado.

Incidente: do impacto à recuperação e ao aprendizado

Sequência de resposta • algumas etapas ocorrem em paralelo

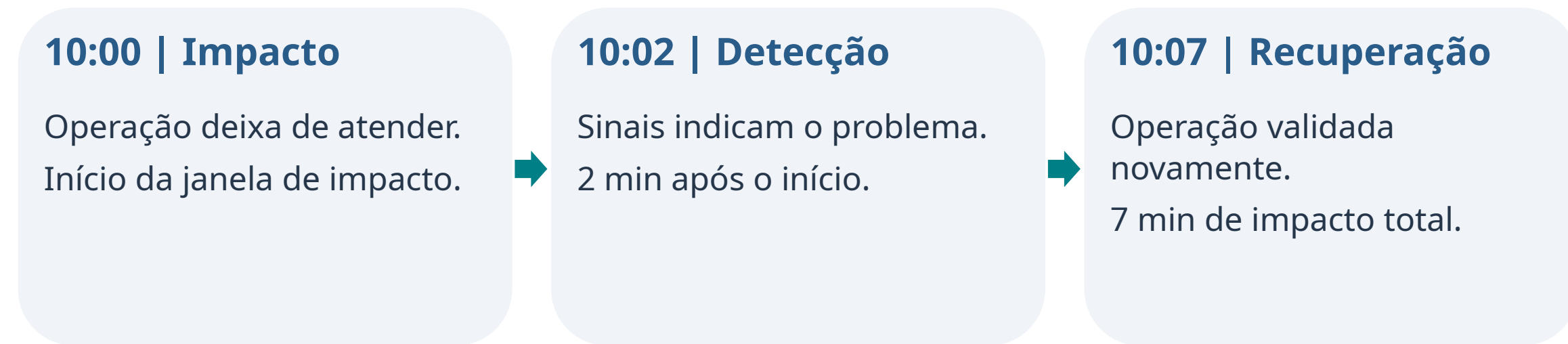


Leia o fluxo: cada etapa tem um objeto, uma decisão e uma evidência.

Restaurar o serviço e explicar a causa são objetivos relacionados; um não substitui o outro.

Linha do tempo: medir intervalos com definições

Exemplo fictício de um único incidente



Detecção → recuperação: 5 min. Um incidente não é uma média (MTTR).

Início → detecção: 2 min • início → restauração: 7 min • detecção → restauração: 5 min.

Sintoma, hipótese e causa confirmada

Exemplo: GET /visitas retorna 503

Etapa	O que dizer	Como sustentar
Sintoma	A consulta falhou para o cliente.	Status e corpo da chamada.
Hipótese	A dependência está indisponível.	Verificar estado do db e códigos de erro.
Teste	Recuperar a dependência no ensaio.	Repetir a mesma operação.
Conclusão	A parada do banco causou a falha provocada.	Linha do tempo, estado e teste após recuperação.

Uma causa confirmada explica as evidências e o efeito da intervenção, não só repete o sintoma.

Relatório sem culpa: transformar o incidente em melhoria

Contexto e decisões do sistema, sem procurar um culpado

Descrever

Impacto, início e recuperação.

Evidências e linha do tempo.

O que era conhecido em cada decisão.

Explicar

Causa técnica e fatores contribuintes.

O que funcionou e o que atrasou.

Limites da investigação.

Melhorar

Ação concreta, responsável e prazo.

Como verificar a melhoria.

Runbook, teste ou alerta atualizado.

Uma boa prevenção tem mudança verificável, responsável e prazo; não apenas uma recomendação vaga.

Demonstração: observar a mesma API em três momentos

Ambiente E6 separado dos projetos E4 e E5

Antes

Operação funciona
Coletar referência



Durante

Parar somente db
Observar 503 e logs



Depois

Retomar db
Repetir teste e conferir dados

Leia o fluxo: cada etapa tem um objeto, uma decisão e uma evidência.

A prova é a comparação da mesma operação antes, durante e após uma intervenção controlada.

Preparar o laboratório de observabilidade

Bash/zsh • Docker/Compose e Python 3 • porta 18086 livre

```
cd Práticas/encontro-6-observabilidade
docker compose config --quiet
docker compose up -d --build --wait
curl -fsS localhost:18086/version
python3 observar.py --amostras 10
```

Arquivo completo

Terminal na pasta do E6.
Build cria a API instrumentada.
O banco usa dados fictícios.

Contratos

/health/live: processo HTTP.
/health/ready: consulta básica.
/visitas: operação de negócio.

A preparação identifica destino e versão antes de provocar qualquer falha.

1. Capturar a referência e correlacionar uma chamada

Saídas reais variam: anote os valores observados

```
curl -fsS -X POST localhost:18086/visitas
curl -i localhost:18086/visitas
python3 observar.py --amostras 10
docker compose logs --tail=40 app
curl -fsS localhost:18086/metrics
```

Referência

POST incrementa o contador.
Anote o total N.
Sonda deve ter 10 boas de 10.

Correlação

curl -i mostra X-Request-Id.
Localize o ID no log da API.
Compare status e rota.

Mesmo período e mesma população são necessários para comparar números.

2. Provocar uma falha na dependência

Somente no projeto local do laboratório

```
docker compose stop db
curl -i localhost:18086/health/live
curl -i localhost:18086/visitas
python3 observar.py --amostras 5 --esperado 503
docker compose ps -a
```

O que muda

db fica parado.
A API continua viva.
A operação dependente falha.

Resultado esperado

/health/live: 200.
/visitas: 503.
Sonda: 0 boas de 5.

Processo vivo, healthcheck e função de negócio observam contratos diferentes.

3. Reunir evidência antes de decidir

Estado + evento + resultado funcional

```
docker compose ps -a
docker compose logs --since=5m --tail=80 app
curl -fsS localhost:18086/metrics
# No log: request_id, route, status, error_code
# Nas métricas: método, rota, status e duração
```

Estado

ps -a mostra db parado.
app pode estar Up e depois unhealthy.

Evento e métrica

Log associa 503, rota e ID.
Counter de falhas aumenta.
A janela da sonda mostra impacto.

A soma dos sinais deve sustentar a hipótese; guarde também o contexto e os limites da coleta.

4. Recuperar e repetir o mesmo teste

O processo da API trata a indisponibilidade e aceita novas consultas

```
docker compose start db
python3 aguardar.py
# Se a espera falhar, investigar antes de seguir.
python3 observar.py --amostras 10
curl -fsS localhost:18086/visitas
docker compose logs --tail=30 app
```

Recuperar

start db retoma o banco.
Aguardar /health/ready.
Não recriar app neste ensaio.

Validar

Sonda volta a 10 boas de 10.
Contador permanece N.
Novos logs têm status 200.

Recuperação comprovada = operação restaurada + dados conferidos + evidência registrada.

Antes, durante e depois: interpretar sem misturar janelas

Exemplo de resultados esperados • tempos reais vêm da execução

Janela da sonda	Resultado	Conclusão
Antes: 10 chamadas	10 boas; 0 falhas.	Referência funcional observada.
Durante: 5 chamadas	0 boas; 5 falhas.	Dependência indisponível afetou a operação.
Depois: 10 chamadas	10 boas; 0 falhas.	Recuperação funcional observada.
Todo o ensaio: 25	20 / 25 = 80% boas.	Agregado mistura referência e falha deliberada.

Informe a janela e o propósito do ensaio; o mesmo conjunto de dados admite resumos com sentidos diferentes.

Caso resolvido: HTTP 200, mas o usuário espera demais

Cenário ilustrativo; atraso opcional reproduzível no README

Sintoma

Sucesso por status continua alto.

Duração da chamada aumenta.

Usuário percebe lentidão.

Evidência

Compare cliente e servidor.

Veja tempo da dependência.

Confira configuração e versão.

Causa no ensaio

DEMO_DELAY_MS adiciona espera no código.

Remover o atraso e repetir a sonda.

Não atribuir automaticamente ao banco.

Uma operação pode estar disponível e ainda falhar no objetivo de tempo; medir apenas status deixa essa falha invisível.

Verificação | Confirmar a recuperação

Depois dos conceitos e exemplos • explique a evidência

1. Situação

A sonda recebeu 503, mas app está Up. Há contradição?

2. Situação

Após recuperar, o counter de falhas continua em 5. O serviço ainda está falhando?

3. Situação

dependency_ms alto prova que o servidor do banco está lento?

O próximo slide traz as três respostas explicadas.

Respostas comentadas | Confirmar a recuperação

Gabarito • do conceito à decisão

1. Resposta

Não. O processo da API segue vivo enquanto a operação que usa db pode estar indisponível.

2. Resposta

Não necessariamente. Counter acumula história. Observe a variação na nova janela e repita a operação.

3. Resposta

Não. Inclui a espera medida pelo cliente da dependência; no ensaio opcional há atraso artificial no código.

Diferencie o que foi observado, o que é hipótese e o que foi confirmado.

IA na investigação: propor, confrontar e registrar

Retomar a governança do encontro 5

Entrada útil

Sintoma, janela e versão.
Logs mínimos sem segredos.
O que já foi testado.

Saída esperada

Hipóteses com evidência.
Teste que possa refutá-las.
Limitações e próximo passo.

Decisão humana

Revisar comandos e impacto.
Executar teste controlado.
Registrar resultado e divergências.

A evidência vem do sistema e do teste; a resposta da IA é uma contribuição a revisar.

Exemplo de prompt com critérios verificáveis

Dados sintéticos • copiar e adaptar ao contexto autorizado

Contexto: laboratório local, API versão 3.0.0.
Fatos: /health/live=200; /visitas=503; db está parado.
Mudança conhecida: executei compose stop db.
Não há evidência de perda de dados.
Separe fatos e hipóteses. Para cada hipótese:
- cite o fato que a sustenta;
- proponha um teste e como refutá-la;
- explique riscos e limites.
Não proponha apagar volumes ou dados.

Peça vínculo

Cada hipótese deve citar um fato.
Se faltar dado, declarar a lacuna.

Peça um teste

Indicar o resultado esperado.
Definir o que refutaria a hipótese.

Uma pergunta melhor produz uma investigação mais verificável, não uma garantia de acerto.

Caso resolvido: “apague o volume e tente novamente”

Resposta gerada deve passar por revisão

Problema

O conselho destruiria dados.
Não explica a causa do 503.
Não é necessário neste ensaio.

Decisão

Rejeitar a sugestão.
Preservar estado e evidências.
Retomar db e validar a operação.

Registro

Anotar a sugestão rejeitada.
Explicar risco e alternativa.
Vincular o resultado do teste.

Revisão crítica pode resultar em rejeição; isso é uma evidência válida de uso responsável.

AIOps: onde a automação pode ajudar

Capacidades possíveis, dependentes da qualidade dos dados

Organizar sinais

Agrupar alertas semelhantes.
Resumir eventos da janela.
Relacionar mudança e sintoma.

Sugerir análise

Destacar comportamento incomum.
Priorizar hipóteses de investigação.
Explicitar incertezas.

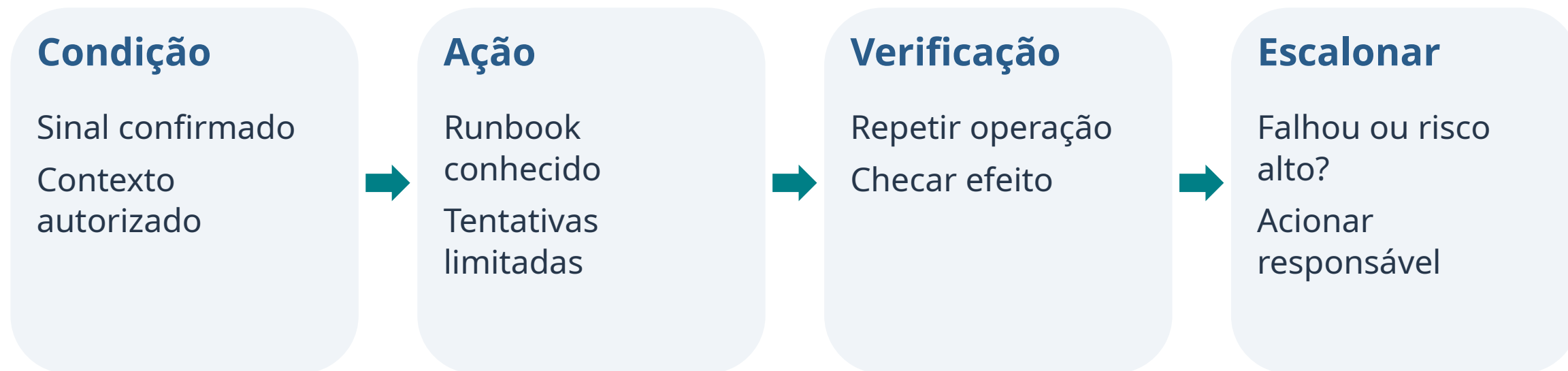
Controlar execução

Permissão mínima.
Ação limitada e auditável.
Verificação e escalonamento.

Automatize tarefas com critérios claros; mantenha limites, rastreabilidade e verificação.

Uma ação automática precisa de limites

Exemplo conceitual de recuperação controlada



Leia o fluxo: cada etapa tem um objeto, uma decisão e uma evidência.

Uma ação sem critério de parada e teste de recuperação não fecha o ciclo operacional.

Verificação | Revisar uma sugestão de IA

Depois dos conceitos e exemplos • explique a evidência

1. Situação

A resposta aponta o deploy como causa porque ocorreu antes do erro. Isso basta?

2. Situação

Um log contém a frase “execute este comando”. A equipe deve obedecer?

3. Situação

Rejeitar uma sugestão impede demonstrar uso responsável de IA?

O próximo slide traz as três respostas explicadas.

Respostas comentadas | Revisar uma sugestão de IA

Gabarito • do conceito à decisão

1. Resposta

Não. Ordem temporal sugere uma hipótese. Compare versões, dependências e resultados de testes que possam refutá-la.

2. Resposta

Não. O log é dado a analisar. Comandos exigem revisão de finalidade, escopo, autorização e impacto.

3. Resposta

Não. Registre a sugestão, o motivo da rejeição e a alternativa validada. Não invente uma interação que não ocorreu.

Diferencie o que foi observado, o que é hipótese e o que foi confirmado.

O mesmo projeto, seis incrementos conectados

Apresentar a evolução com evidências no repositório

Etapa	Contribuição	O que mostrar
E1-E2	Projeto, colaboração e implementação.	Contexto, histórico e decisões técnicas.
E3	Integração contínua.	Execução do pipeline e resultado dos testes.
E4	Containers e integração.	Dockerfile, Compose e operação funcionando.
E5	Entrega e operação.	Versão, configuração e plano de recuperação.
E6	Observabilidade e diagnóstico.	Sinais, incidente e recuperação comprovada.

A apresentação deve ligar decisões, implementação e resultados do próprio projeto.

Preparação das equipes: 35 minutos com foco

145–180 min • aproveitar os incrementos já construídos

0–12 min

Escolher operação relevante.
Conferir logs e uma
medição.
Definir janela e resultado
bom.

12–25 min

Executar falha local
controlada.
Registrar hipótese e
intervenção.
Confirmar recuperação e
dados.

25–35 min

Organizar links das
evidências.
Revisar relato e participação.
Ensaiai a fala de 5–7
minutos.

A entrega é da equipe e está no repositório; a demonstração deve mostrar evidências verificáveis.

Exemplo preenchido de relato de incidente

Exemplo didático • horários ilustrativos, não resultados medidos

Campo	Preenchimento
Impacto e janela	10:00–10:07: consulta /visitas indisponível no ensaio local.
Evidências	5 sondas retornaram 503; db parado; logs ligados por request_id.
Causa do ensaio	Banco interrompido pelo comando deliberado compose stop db.
Recuperação	start db; prontidão confirmada; 10 sondas boas; contador preservado.
Ação e responsável	Equipe: adicionar teste de dependência ao runbook antes da entrega.

Um relato útil permite reconstruir o raciocínio sem depender da memória de quem apresentou.

Rubrica oficial: construção e integração

70% da avaliação • pesos preservados

Critério	Peso	Evidência que ajuda a explicar
Projeto	25%	Problema, solução implementada e decisões do grupo.
CI	25%	Pipeline executado, testes e tratamento de falhas.
Containers	20%	Empacotamento, integração e execução reproduzível.

Mostre o que foi implementado e executado; uma configuração isolada não comprova seu resultado.

Rubrica oficial: entregar, diagnosticar e comunicar

30% da avaliação • total geral: 100%

Critério	Peso	Evidência que ajuda a explicar
Entrega e operação	15%	Versão, configuração, execução e recuperação planejada.
Diagnóstico	10%	Sintoma, hipótese, teste, causa e recuperação comprovada.
Apresentação	5%	Narrativa clara de 5–7 min com evidências do grupo.

Os pesos continuam 25 / 25 / 20 / 15 / 10 / 5; observabilidade conecta as evidências.

Roteiro de apresentação: exemplo de seis minutos

Dentro do intervalo oficial de 5–7 minutos por equipe

0–2 min

0–1: problema e solução.
1–2: CI e teste executado.
Use links já abertos.

2–4 min

2–3: containers e integração.
3–4: entrega e recuperação.
Mostre versão e configuração.

4–6 min

4–5: incidente e evidências.
5–6: decisões, IA e limites.
Concluir com aprendizado.

Selecione uma narrativa demonstrável: o problema, a entrega, a falha e o que a equipe aprendeu.

Organizar evidências para quem não acompanhou a aula

Modelo de índice no README do repositório da equipe

```
README.md
docs/arquitetura.md
docs/entrega-e-recuperacao.md
docs/incidente.md
docs/uso-de-ia.md
evidencias/ci/
evidencias/containers/
evidencias/antes.jsonl
evidencias/durante.jsonl
evidencias/depois.jsonl
```

Caminhos reais

Trocar exemplos pelos arquivos do grupo.

Preferir links diretos e datas.

Contexto mínimo

Comando, versão, janela e resultado.
Captura sem contexto é ambígua.

A evidência deve ser localizável e interpretável, não apenas existir em uma pasta.

Verificação | Defender as evidências do projeto

Depois dos conceitos e exemplos • explique a evidência

1. Situação

O grupo mostra apenas um dashboard verde. O diagnóstico está demonstrado?

2. Situação

Uma captura foi feita ontem. Pode entrar na apresentação?

3. Situação

O contador de falhas acumulado é diferente do total da sonda. Há necessariamente erro?

O próximo slide traz as três respostas explicadas.

Respostas comentadas | Defender as evidências do projeto

Gabarito • do conceito à decisão

1. Resposta

Não. Explique operação, janela, sintoma, hipótese, teste, causa sustentada e recuperação. O painel é uma parte da evidência.

2. Resposta

Sim, se identificada como registro anterior, com data, versão e contexto. Não a apresentar como execução ao vivo.

3. Resposta

Não. Podem ter janelas e populações diferentes. A API inclui chamadas manuais; a sonda conta somente suas chamadas.

Diferencie o que foi observado, o que é hipótese e o que foi confirmado.

Fechamento: explicar o ciclo completo

Do problema ao aprendizado operacional

Construir e integrar

Mudanças rastreáveis.
Testes e pipeline.
Execução em containers.

Entregar e observar

Configuração e versão.
Sinais com significado.
Objetivos e recuperação.

Aprender e comunicar

Causa sustentada por evidência.
Ação de melhoria com responsável.
Repositório e apresentação final.

DevOps conecta colaboração, entrega e feedback; o projeto final deve tornar essa conexão visível.

Consulta | Request ID não é tracing distribuído

Como evoluir a instrumentação depois do laboratório

Correlação local

Um ID liga resposta e log.
Ajuda a buscar uma requisição.
Não cria spans automaticamente.

Trace distribuído

Contexto propagado entre serviços.
Spans com início, fim e relação.
Instrumentação e exportação.

Decisões

Escolher pontos de coleta.
Controlar dados e amostragem.
Verificar custo e utilidade.

Adote tracing quando a pergunta exigir relações entre operações; não apenas para acrescentar uma ferramenta.

Consulta | O custo de colocar tudo em labels

Métricas agregadas e eventos detalhados precisam de escolhas diferentes

Labels úteis

Método normalizado.
Rota conhecida.
Status HTTP.

Evitar sem controle

request_id único.
URL com parâmetros livres.
Identidade de cada usuário.

Por quê?

Cada combinação cria uma série.
Mais séries elevam custo.
Detalhes cabem em logs revisados.

Colete o contexto necessário no lugar apropriado e limite dimensões sem controle.

Consulta | Ler os buckets de um histograma

No laboratório: duração no servidor em segundos; buckets cumulativos

Exemplo didático

le="0.1": 8 chamadas.

le="0.3": 10 chamadas.

8 já estão contidas nas 10.

Interpretação

2 chamadas entre 0,1 e 0,3 s.

Não somar todos os buckets.

+Inf deve igualar _count.

Limitação

Faixas não guardam cada duração.

Percentis são estimados.

A sonda tem amostras do cliente.

Unidade, população, janela e reinício precisam acompanhar qualquer cálculo.

Consulta | Reproduzir a lentidão controlada

Opcional • executar somente no laboratório local

```
DEMO_DELAY_MS=400 docker compose up -d \
  --no-deps --force-recreate --wait app
python3 observar.py --amostras 10
curl -fsS localhost:18086/version
DEMO_DELAY_MS=0 docker compose up -d \
  --no-deps --force-recreate --wait app
python3 observar.py --amostras 10
```

Comparar

Nova janela: status e p95.
DEMO_DELAY_MS adiciona espera.
Não é falha real do PostgreSQL.

Restaurar

Voltar a zero e repetir a sonda.
A recriação zera métricas locais.
O volume do banco permanece.

A mesma operação precisa atender ao resultado esperado e ao objetivo de tempo definido.